

CareerForge AI

Project Report

*A Full-Stack Career Platform for Resume Building,
Portfolio Building, and AI-Powered Resume Review*

Submitted by

Amit Singh Bisht

TuxAcademy Internship Program

GitHub: github.com/amitbisht2406-cmd/fresh

Table of Contents

1. Abstract.....	3
2. Problem Statement.....	3
3. Objectives.....	3
4. Scope of the Project.....	3
5. System Requirements.....	3
5.1 Software Requirements.....	3
5.2 Hardware Requirements.....	4
6. Technology Stack.....	4
7. System Design.....	4
7.1 System Architecture.....	4
7.2 Entity-Relationship Diagram.....	5
7.3 User Flow / Process Flowchart.....	6
8. Module Description.....	6
9. Database Design.....	7
10. API Endpoints Summary.....	7
11. Future Scope.....	8
12. Conclusion.....	8
13. References.....	8

1. Abstract

CareerForge AI is a full-stack web platform built to help job seekers, particularly students and early-career professionals, prepare for the job market in one place. The platform combines three tools that are normally scattered across separate apps: a guided resume builder with live preview and one-click PDF export, a portfolio builder for showcasing projects online, and an AI-powered resume reviewer that scores a resume for overall quality, grammar, ATS compatibility, and skills coverage. The system is built with an Angular frontend, an ASP.NET Core Web API backend, a SQL Server database, and the Groq API (Llama 3.3 70B) for AI-driven feedback.

2. Problem Statement

Job seekers often struggle to build a well-structured, ATS-friendly resume without design or formatting experience, and separately need a portfolio to showcase projects alongside that resume. On top of that, most people have no easy way to get objective feedback on a resume before submitting job applications. The tools that exist today typically solve only one of these problems in isolation, which forces users to juggle multiple disconnected platforms just to get application-ready. CareerForge AI addresses this by bringing resume building, portfolio building, and AI-driven resume review into a single, unified dashboard.

3. Objectives

- Provide a guided, multi-section resume builder covering personal information, education, skills, projects, certificates, and languages.
- Allow users to build and preview a personal portfolio (hero section, about, skills, projects, contact).
- Generate instant, AI-based feedback on resume quality using an external AI service.
- Support secure, authenticated access to each user's own data using JWT.
- Offer a responsive interface that adapts from desktop to mobile.
- Allow one-click export of a finished resume as a PDF document.

4. Scope of the Project

The current scope covers the core workflow of registering an account, building a resume and/or portfolio, receiving AI feedback on a resume, exporting the resume as a PDF, and managing saved work from a central dashboard and downloads area. Multi-resume support, public shareable portfolio links, and production deployment with CI/CD are treated as future scope rather than part of the current release (see Section 11).

5. System Requirements

5.1 Software Requirements

Component	Requirement
Frontend Runtime	Node.js, Angular CLI
Backend Runtime	.NET SDK (ASP.NET Core)
Database	Microsoft SQL Server + SQL Server Management Studio

AI Service	Groq API key
Version Control	Git & GitHub
API Testing	Postman

5.2 Hardware Requirements

- A development machine capable of running Visual Studio / VS Code, SQL Server, and Node.js simultaneously (minimum 8 GB RAM recommended).
- Internet access for Groq API calls and package installation.

6. Technology Stack

Layer	Technologies Used
Frontend	Angular (standalone components, Reactive Forms, SSR), TypeScript, html2canvas, jsPDF
Backend	ASP.NET Core Web API, Entity Framework Core, JWT Bearer Authentication, BCrypt.Net
Database	Microsoft SQL Server
AI / ML	Groq API — Llama 3.3 70B (resume scoring and feedback)
Dev Tools	Visual Studio, Visual Studio Code, SSMS, Postman, Git & GitHub

7. System Design

7.1 System Architecture

The system follows a classic three-tier architecture. The Angular single-page application in the client tier communicates over HTTPS/REST with an ASP.NET Core Web API in the application tier. The API layer is organized into controllers (Auth, Users, Resumes, Portfolios, AI), protected by JWT bearer authentication and BCrypt password hashing, and talks to SQL Server through Entity Framework Core. For AI resume review, the API layer calls out to the external Groq API (Llama 3.3 70B) and returns the score and feedback to the client.

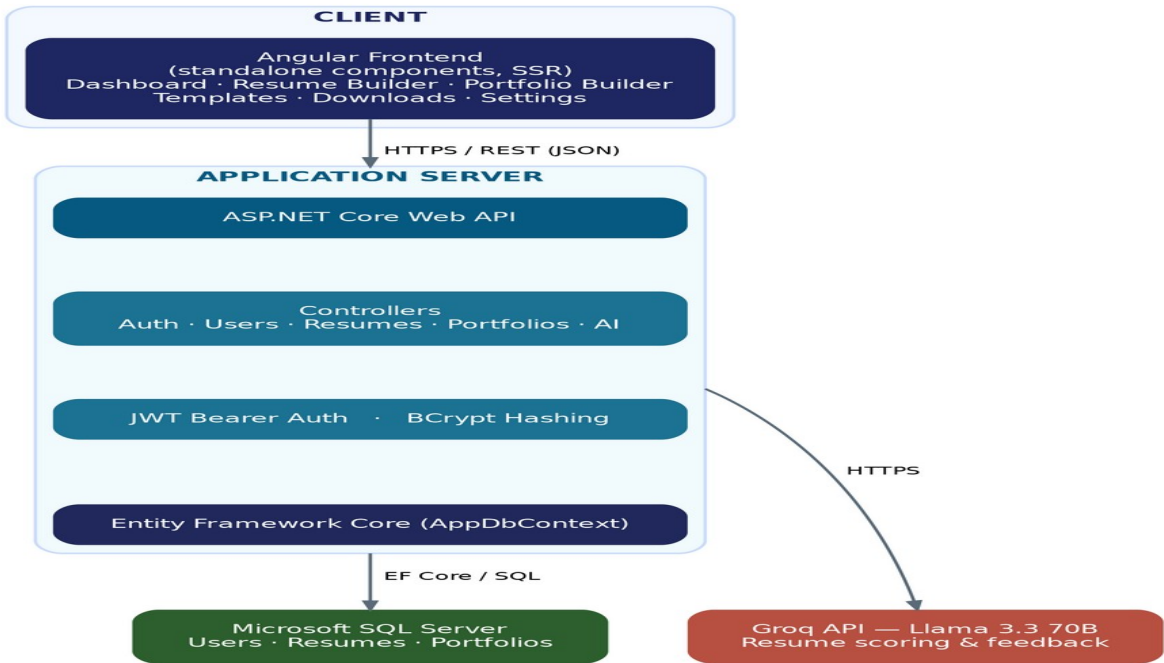


Figure 1: System Architecture Diagram

7.2 Entity-Relationship Diagram

The data model centers on three entities. A User can create many Resumes and many Portfolios (one-to-many relationships). Resume and Portfolio store repeating sub-sections such as skills, projects, certificates, and languages as JSON-encoded text columns rather than as separate normalized tables, which keeps the schema simple while still allowing flexible, form-driven content.

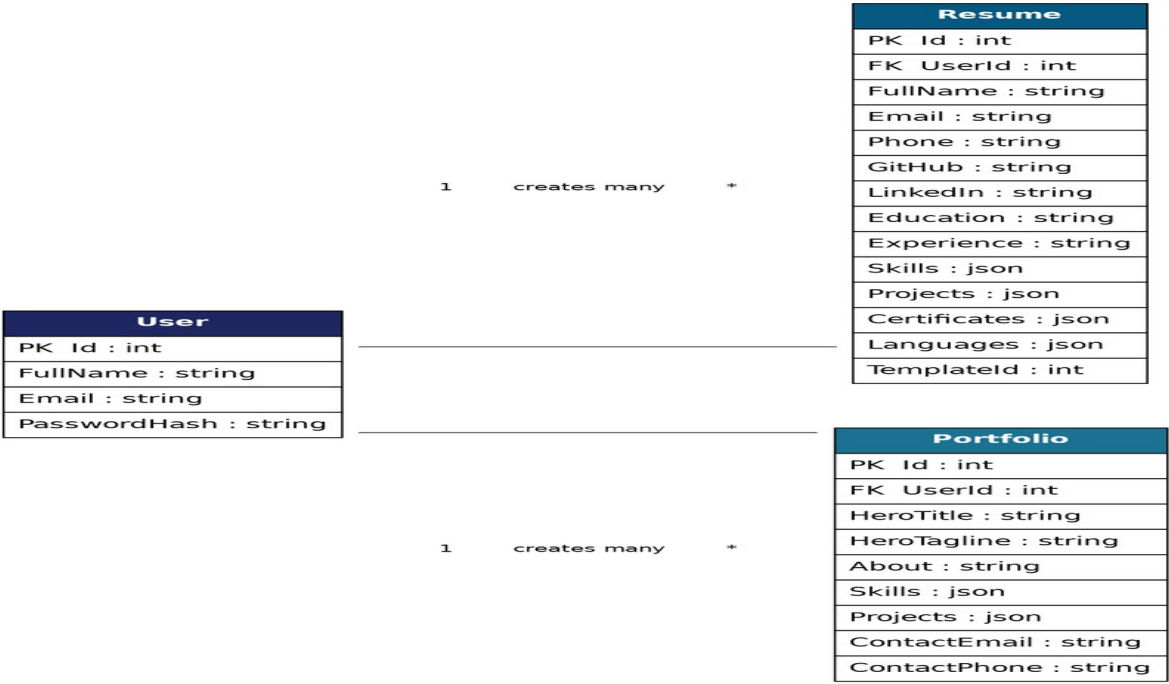


Figure 2: Entity-Relationship Diagram

7.3 User Flow / Process Flowchart

A new visitor registers or logs in, lands on the dashboard, and chooses either the Resume Builder or the Portfolio Builder. Resume creation additionally allows choosing a template. Both builders feed into a live preview; from the resume preview, the user can trigger the AI Resume Review (Groq-powered) and then export the finished resume as a PDF. Every save or download action raises an in-app notification and appears afterward in the Downloads center.

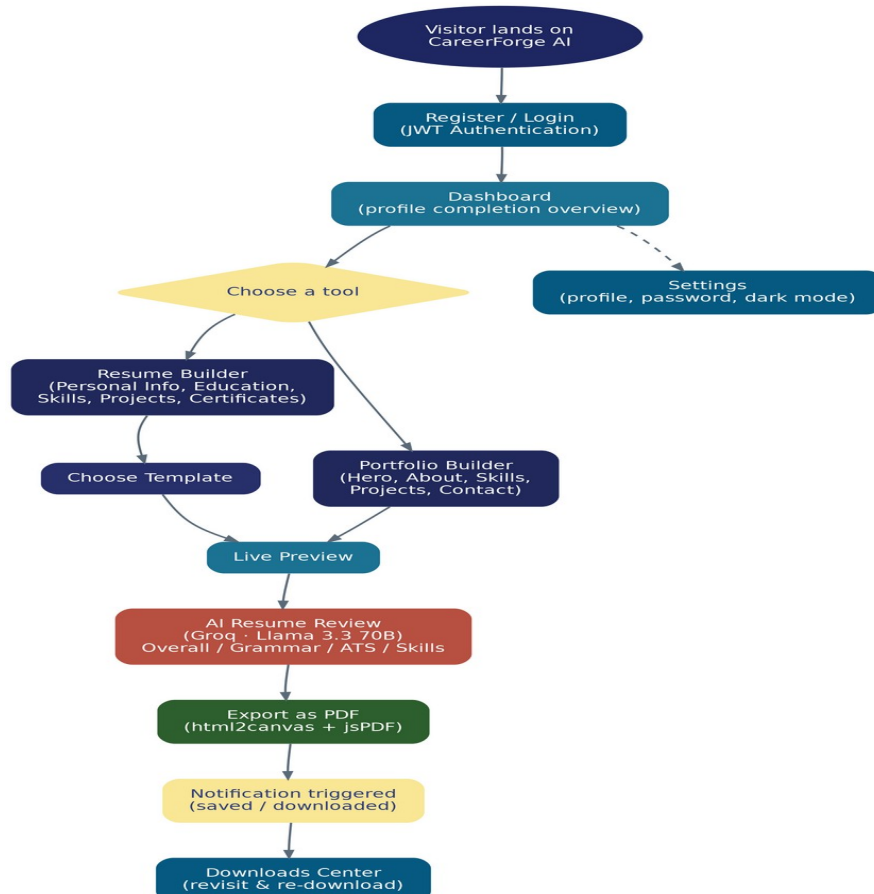


Figure 3: User Journey Flowchart

8. Module Description

8.1 Authentication Module

Handled by AuthController, exposing register and login endpoints. Passwords are hashed with BCrypt before storage; successful login issues a JWT used to authorize subsequent requests.

8.2 User Module

UsersController exposes endpoints to fetch the current profile (GET /api/users/me), update profile details, and change the account password.

8.3 Resume Module

ResumesController provides full CRUD (create, read, update, delete) for a user's resumes. The Resume model captures personal details (name, email, phone, GitHub, LinkedIn), education, experience, and JSON-encoded lists for skills, projects, certificates, and languages, along with a selected TemplateId.

8.4 Portfolio Module

PortfoliosController mirrors the resume module's CRUD pattern for portfolios. The Portfolio model stores a hero title and tagline, an about section, JSON-encoded skills and projects, and contact details (email, phone).

8.5 AI Review Module

AiController exposes a review-resume endpoint that forwards resume content to GroqService, which calls the Groq API (Llama 3.3 70B model) and returns a structured score (Overall, Grammar, ATS Compatibility, Skills) plus written feedback.

8.6 Dashboard, Downloads & Settings

On the frontend, the Dashboard summarizes profile completion and links into each tool. Downloads centralizes previously generated resumes and portfolios for re-access. Settings covers profile editing, password changes, and the app-wide dark mode toggle, which persists across sessions.

9. Database Design

The database uses three primary tables mapped from the C# models below via Entity Framework Core migrations.

Table	Key Columns
Users	Id (PK), FullName, Email, PasswordHash
Resumes	Id (PK), UserId (FK), FullName, Email, Phone, GitHub, LinkedIn, Education, Experience, Skills (JSON), Projects (JSON), Certificates (JSON), Languages (JSON), TemplateId
Portfolios	Id (PK), UserId (FK), HeroTitle, HeroTagline, About, Skills (JSON), Projects (JSON), ContactEmail, ContactPhone

10. API Endpoints Summary

Controller	Endpoints
AuthController	POST /api/auth/register, POST /api/auth/login
UsersController	GET /api/users/me, PUT /api/users/me, PUT /api/users/me/password
ResumesController	GET/POST /api/resumes, PUT/DELETE /api/resumes/{id}
PortfoliosController	GET/POST /api/portfolios, PUT/DELETE /api/portfolios/{id}
AiController	POST /api/ai/review-resume

11. Future Scope

- Support for multiple resumes and portfolios per user, with the ability to switch between them.
- Public, shareable portfolio links that don't require login to view.
- AI-powered resume rewriting, going beyond scoring into actual content generation.
- An AI career roadmap generator that performs skill-gap analysis and suggests a learning path.
- Additional resume template designs with visual switching.
- Push notifications, in addition to the current in-app notification center.
- Production deployment with a CI/CD pipeline.

12. Conclusion

CareerForge AI demonstrates a complete, end-to-end full-stack workflow: secure authentication, structured data entry across two distinct builders, persistence through Entity Framework Core and SQL Server, and an AI integration that gives users actionable feedback rather than a static template. The project brings together frontend, backend, database, and third-party AI service integration into a single cohesive product, and lays a clear foundation for the future enhancements identified above.

13. References

Angular Documentation — angular.dev

ASP.NET Core Documentation — learn.microsoft.com/aspnet/core

Entity Framework Core Documentation — learn.microsoft.com/ef/core

Groq API Documentation — console.groq.com/docs

Project Repository — github.com/amitbisht2406-cmd/fresh